

Hotspot Analysis

Andrea Cattarinich S5137057

November 10, 2025

1 Hotspot Analysis

The goal of this analysis is to identify the parts of the program that consume the most execution time. The measurements were performed using two methods:

- **Manually:** using `omp_get_wtime()`
- **Profiler:** using Intel VTune

Either two methods have been used with with $[N = 40000]$ on a Room-210 Workstation and produced similar results.

Table 1: Manually

Function	Time (s)	Percentage (%)
fillInput	0.0053	0.03
setOutputZero1	0.0005	0.00
setOutputZero2	0.0005	0.00
DFT	8.0331	49.97
IDFT	8.0345	49.99
checkResults	0.0000	0.00
Total	16.0741	100.00

Table 2: Intel VTune Profiler

Function Stack	CPU Time: Self	Percentage (%)
Total	16.06	100.00
._start	16.06	0.00
._libc_start_main_impl	16.06	0.00
- main	16.06	0.00
— DFT	8.032	14.21
— _svml_sincos2	5.74977	1.72
— _svml_sincos2_l9	5.47389	34.08
— DFT	8.028	14.02
— _svml_sincos2	5.77596	1.54
— _svml_sincos2_l9	5.52797	34.42

1.1 Conclusion

The main hotspot is the `DFT()` function and its inverse `IDFT()`, which together account for over 99% of the total execution time. All other functions have negligible impact. This indicates that **parallelization and vectorization efforts should focus on the double loop inside `DFT()`**.

In particular, the main hotspot is the loop line contained in lines 103-112, as shown in the following screenshot.

Intel VTune Profiler

Project Naviga... +

homework1

r000hs

r001hs

Welcome x r001hs x

Hotspots

Summary

Bottom-up

Caller/Callee

Top-down Tree

Flame Graph

Platform

omp_homework_hotspots_v2.c x omp_homework_hotspots_v2.c x

Source Assembly

Source Line ▲	Source	CPU Time: Total	CPU Time: Self
91			
92			
93	end = omp_get_wtime();		
94	printf("Total time: %.10f seconds\n", end-start_entire_program);		
95			
96	return 1;		
97	}		
98			
99	// DFT/IDFT routine		
100	// idft: 1 direct DFT, -1 inverse IDFT (Inverse DFT)		
101	int DFT(int idft, double* xr, double* xi, double* Xr_o, double* Xi_o, int N){		
102	int k, n;		
103	for (k=0 ; k<N ; k++){	0.5%	0.080s
104	{		
105	for (n=0 ; n<N ; n++) {		
106	// Real part of X[k]		
107	Xr_o[k] += xr[n] * cos(n * k * PI2 / N) + idft*xi[n]*sin(n * k * PI	5.4%	0.866s
108	// Imaginary part of X[k]		
109	Xi_o[k] += -idft*xr[n] * sin(n * k * PI2 / N) + xi[n] * cos(n * k *	44.1%	1.336s
110			
111	}		
112	}		
113			
114	// normalize if you are doing IDFT		
115	if (idft==1){		
116	for (n=0 ; n<N ; n++){		
117	Xr_o[n] /=N;		